

LAPORAN MILESTONE 1 TUGAS BESAR IF2224 TEORI BAHASA FORMAL DAN OTOMATA

ARION COMPILER

Lexical Analysis

DEADLINE: Kamis, 2 April 2026 Pukul 23:59 WIB



Dipersiapkan oleh:

Kelompok AZZAM | Kode MAR | Kelas 01

13524017	Aziza Dharma Putri
13424053	Ahmad Zaky Robbani
13524063	Marcel Luther Sitorus
13524081	Alya Nur Rahmah

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132
2026**

DAFTAR ISI

DAFTAR ISI.....	2
TEORI SINGKAT.....	3
1.1. DFA.....	3
1.2. Analisis Leksikal.....	5
1.3. Token.....	5
PERANCANGAN DAN IMPLEMENTASI.....	6
2.1. Perancangan.....	6
2.2. Perancangan DFA.....	6
2.3. Perancangan Analisis Lexer.....	8
2.4. Implementasi.....	8
2.4.1. Alur Program.....	8
2.4.2. Kode Program Utama.....	10
2.4.3. Kode Kelas Token.....	12
2.4.4. Kode Kelas DFA.....	14
2.4.5. Kode Kelas Lexer.....	19
2.4.6. Config.....	29
PENGUJIAN.....	31
3.1. Pengujian Pertama.....	31
3.2. Pengujian Kedua.....	32
3.3. Pengujian Ketiga.....	33
3.4. Pengujian Keempat.....	36
3.5. Pengujian Kelima.....	36
3.6. Pengujian Keenam.....	39
3.7. Pengujian Ketujuh.....	41
3.8. Pengujian Kedelapan.....	43
3.9. Pengujian Kesembilan.....	43
3.10. Pengujian Kesepuluh.....	45
3.11. Pengujian Kesebelas.....	46
3.12. Pengujian Keduabelas.....	47
3.13. Pengujian Ketigabelas.....	48
3.14. Pengujian Keempatbelas.....	49
3.15. Pengujian Kelimabelas.....	50
3.16. Pengujian Otomatis.....	51
KESIMPULAN DAN SARAN.....	52
4.1. Kesimpulan.....	52
4.2. Saran.....	52
LAMPIRAN.....	53
DAFTAR PUSTAKA.....	54

BAB I

TEORI SINGKAT

1.1. DFA

1.1.1. Definisi Formal DFA

Deterministic Finite Automaton (DFA) secara formal didefinisikan sebagai 5-tuple berikut ini :

$$A = (Q, \Sigma, \delta, q_0, F)$$

dengan komponen-komponen sebagai berikut:

1. **Q** adalah himpunan berhingga dari state (keadaan). Setiap state merepresentasikan kondisi tertentu dari mesin pada suatu titik selama pemrosesan masukan. Dalam konteks lexer, state-state ini merepresentasikan progres pembacaan suatu token, misalnya state untuk membaca digit angka, state untuk membaca karakter dalam string, dan sebagainya.
2. **Σ** (sigma) adalah himpunan berhingga dari simbol masukan yang disebut alfabet. Alfabet ini mendefinisikan seluruh karakter yang dapat diproses oleh automaton. Pada lexer Arion, alfabet mencakup huruf (a–z, A–Z), digit (0–9), simbol-simbol operator (+, -, *, /, <, >, =), tanda baca (;, :, ., ,), tanda kurung ((,), [,], {, }), tanda petik tunggal ('), underscore (_), serta karakter whitespace.
3. **δ** (delta) adalah fungsi transisi yang memetakan setiap pasangan state dan simbol masukan ke tepat satu state berikutnya. Secara formal, $\delta: Q \times \Sigma \rightarrow Q$. Sifat deterministik inilah yang menjadi ciri khas DFA — untuk setiap state dan setiap simbol masukan, selalu terdapat tepat satu state tujuan yang unik. Hal ini menjamin bahwa proses komputasi selalu jelas dan tidak ambigu.
4. **q_0** adalah state awal (*start state*), yaitu state tempat automaton memulai pemrosesan masukan. Berlaku $q_0 \in Q$. Pada diagram transisi DFA, state awal ditandai dengan anak panah masuk yang tidak berasal dari state manapun.
5. **F** adalah himpunan state akhir atau state penerima (*accepting states*). Berlaku $F \subseteq Q$. Jika automaton berada pada salah satu state dalam F setelah seluruh masukan selesai diproses, maka string masukan tersebut diterima (*accepted*). Pada diagram transisi, state penerima digambar dengan lingkaran ganda.

1.1.2. Bahasa yang Diterima DFA

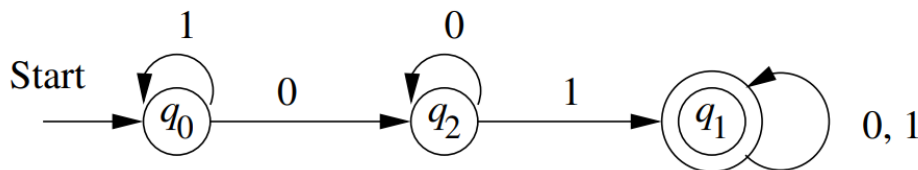
Bahasa yang diterima (atau dikenali) oleh suatu DFA A, dinotasikan sebagai $L(A)$, adalah himpunan seluruh string atas alfabet Σ yang membawa automaton dari state awal ke salah satu state penerima (Hopcroft et al., 2007). Secara formal:

$$L(A) = \{w \mid \hat{\delta}(q_0, w) \text{ is in } F\}$$

Suatu string w diterima jika dan hanya jika ketika automaton memulai dari state awal q_0 dan memproses seluruh simbol dalam w satu per satu, automaton berakhir di salah satu state dalam himpunan F. Sebaliknya, jika automaton berakhir di state yang bukan anggota F, maka string tersebut ditolak.

Bahasa-bahasa yang dapat dikenali oleh DFA (dan NFA) disebut bahasa reguler (*regular languages*). Kelas bahasa reguler ini ekuivalen dengan kelas bahasa yang dapat dideskripsikan oleh ekspresi reguler (*regular expressions*). Keekuivalenan ini memberikan fleksibilitas dalam perancangan, karena pola token dapat didefinisikan terlebih dahulu menggunakan ekspresi reguler, kemudian dikonversi menjadi DFA untuk implementasi.

1.1.3. Diagram Transisi



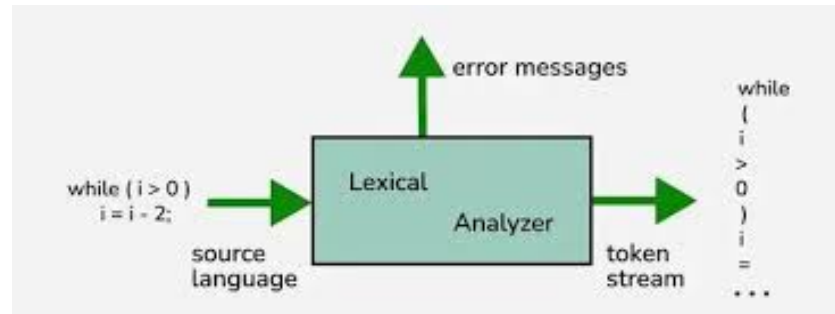
Gambar 1.1.3 Contoh Diagram Transisi DFA

Diagram transisi adalah representasi grafis dari suatu DFA yang mempermudah visualisasi dan pemahaman perilaku automaton (Hopcroft et al., 2007). Elemen-elemen diagram transisi meliputi:

- Node (lingkaran) merepresentasikan state. Setiap state diberi label nama unik (misalnya q_0 , q_1 , q_{37} , dsb.).
- Panah berlabel antar node merepresentasikan transisi. Label pada panah menunjukkan simbol masukan yang memicu transisi tersebut.
- Lingkaran ganda menandai state penerima (*accepting state*), yaitu state yang termasuk dalam himpunan F .
- Panah tanpa asal menunjuk ke state awal q_0 , menandakan titik mulai pemrosesan.
- Self-loop adalah panah yang mengarah kembali ke state yang sama, menandakan bahwa simbol tertentu tidak mengubah state (misalnya membaca digit berturut-turut saat membaca bilangan).

Dalam diagram transisi DFA untuk lexer Arion, setiap jalur dari state awal ke state penerima merepresentasikan pola pengenalan suatu jenis token. Misalnya, jalur yang melewati state-state pembacaan digit merepresentasikan pengenalan token `intcon`, sedangkan jalur melalui state-state pembacaan huruf merepresentasikan pengenalan token `ident` atau `keyword`.

1.2. Analisis Leksikal



Gambar 1.2. Analisis Leksikal

Analisis leksikal merupakan tahap pertama dalam proses kompilasi suatu bahasa pemrograman. Pada tahap ini, kode sumber yang berupa rangkaian karakter mentah dibaca dan diubah menjadi rangkaian unit-unit bermakna terkecil yang disebut token (Hopcroft et al., 2007). Komponen yang melaksanakan tugas ini disebut lexer atau lexical analyzer.

Proses analisis leksikal bekerja dengan cara membaca kode sumber dari kiri ke kanan, karakter demi karakter, dan mengenali pola-pola yang membentuk token. Proses ini menerapkan prinsip *longest match* — lexer selalu berusaha mencocokkan pola terpanjang yang valid. Sebagai contoh, ketika menemukan karakter `<` diikuti `=`, lexer akan mengenalinya sebagai satu token `<=` (leq), bukan dua token `<` (lss) dan `=` secara terpisah.

Selain menghasilkan token, lexer juga bertanggung jawab untuk membuang elemen-elemen yang tidak relevan bagi tahap selanjutnya, seperti whitespace (spasi, tab, newline) dan komentar. Lexer juga berperan dalam mendeteksi kesalahan leksikal, yaitu karakter atau urutan karakter yang tidak sesuai dengan pola token manapun yang valid.

Hubungan antara analisis leksikal dan finite automata sangat erat. Pola-pola token pada dasarnya merupakan bahasa reguler yang dapat dikenali oleh DFA. Oleh karena itu, DFA menjadi model implementasi yang natural dan efisien untuk lexer. Setiap state dalam DFA merepresentasikan progres pengenalan suatu token, dan setiap transisi merepresentasikan konsumsi satu karakter masukan.

1.3. Token

Token merupakan unit leksikal terkecil yang memiliki makna dalam suatu bahasa pemrograman. Setiap token terdiri dari dua komponen utama:

- Jenis token (*token type*): mengkategorikan token berdasarkan perannya dalam bahasa, misalnya keyword, identifier, operator, literal, atau delimiter.
- Nilai token (*token value* atau *lexeme*): rangkaian karakter aktual dari kode sumber yang membentuk token tersebut. Tidak semua token memerlukan nilai misalnya, keyword dan operator sudah sepenuhnya ditentukan oleh jenisnya saja.

BAB II

PERANCANGAN DAN IMPLEMENTASI

2.1. Perancangan

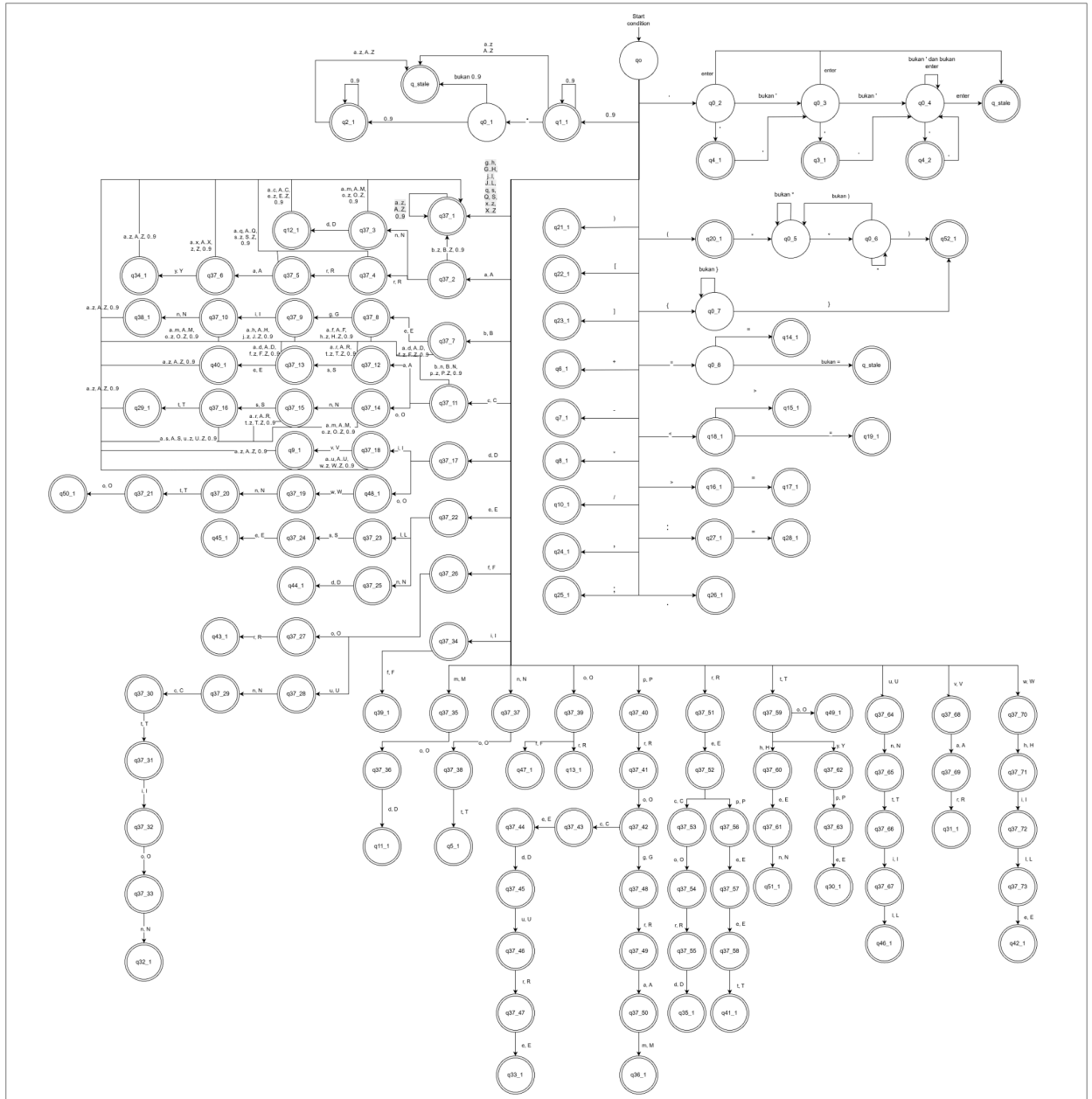
Pada tahap perancangan ini, sistem yang dibangun adalah sebuah lexical analyzer (lexer) untuk bahasa pemrograman Arion yang diimplementasikan dengan menggunakan bahasa pemrograman C++. Sesuai dengan spesifikasi tugas, lexer dirancang untuk membaca source code dalam bentuk file .txt, kemudian mengubahnya menjadi daftar token yang sesuai dengan aturan bahasa Arion.

Pemilihan bahasa C++ dalam perancangan ini didasarkan pada kebutuhan implementasi yang efisien serta kontrol penuh terhadap proses pembacaan karakter. Selain itu, C++ memungkinkan pembuatan program yang modular sesuai dengan anjuran spesifikasi, sehingga komponen lexer dapat dipisahkan dari komponen lain seperti parser atau semantic analyzer pada milestone berikutnya. Selain itu juga, C++ memiliki kinerja yang cepat dan dibutuhkan terutama untuk pemrosesan input berukuran besar pada tugas kali ini.

2.2. Perancangan DFA

Rancangan *Deterministic Finite Automata* (DFA) pada gambar di bawah ini dibuat untuk merepresentasikan proses analisis leksikal dalam mengenali berbagai jenis token dari input yang diberikan. DFA ini dimulai dari *start state* (q_0) yang berfungsi sebagai titik awal pembacaan karakter, kemudian berpindah antar state berdasarkan simbol input yang diterima sesuai dengan aturan transisi yang telah didefinisikan. Setiap jalur transisi dirancang untuk mengakomodasi pola tertentu, seperti huruf, angka, simbol, maupun kombinasi karakter lainnya yang membentuk token valid.

Struktur DFA ini terdiri dari sejumlah state yang saling terhubung, di mana beberapa di antaranya merupakan *final state* yang menandakan bahwa suatu rangkaian input telah berhasil dikenali sebagai token tertentu. Selain itu, terdapat pula state khusus seperti *error state* (misalnya q_{state}) yang digunakan untuk menangani input yang tidak sesuai dengan pola yang diharapkan. Rancangan ini juga memperlihatkan pemisahan jalur untuk berbagai kategori token, sehingga setiap jenis token diproses melalui alur state yang spesifik.



Untuk mendapatkan kualitas gambar lebih jelas dapat diakses [di sini](#).

Pada implementasi ini, kami menggabungkan tipe token COMMENT sebagai string karakter yang di ujungnya diapit (* dan *), atau diapit { dan }. Hal ini dilakukan karena token-token yang berada di antara delimiter komentar tidak akan memiliki makna kedepannya dan penggabungannya menjadi satu token yang dibuang akan mengurangi beban parsing, kemudian jika tidak digabungkan menjadi satu token, string yang berada di antara delimiter comment harus terikat dengan bahasa yang secara normal diterima DFA karena harus berupa token valid. Sedangkan, secara kegunaan, string yang dapat ditempatkan sebagai komentar lebih beragam dan tidak terikat aturan.

2.3. Perancangan Analisis Lexer

Analisis Lexer yang dirancang pada tugas ini menggunakan pendekatan Deterministic Finite Automata (DFA) sebagai dasar utama dalam proses pengenalan token. Sesuai spesifikasi, program lexer harus bekerja layaknya DFA, yaitu membaca karakter input satu per satu dan menentukan transisi state berdasarkan karakter tersebut.

Secara umum, alur kerja lexer adalah sebagai berikut:

1. Program membaca file source code karakter demi karakter.
2. Setiap karakter menjadi input bagi DFA untuk menentukan state berikutnya.
3. Selama masih terdapat transisi yang valid, karakter akan digabungkan menjadi sebuah lexeme.
4. Ketika tidak ada transisi lanjutan, maka lexeme akan diperiksa pada dua kondisi, jika berada pada state final, maka lexeme akan dikonversi menjadi token. Kemudian, jika tidak, maka dianggap sebagai error leksikal.

Pendekatan ini memastikan bahwa lexer mengikuti prinsip DFA secara deterministik tanpa backtracking. Perancangan lexer yang telah dibuat menjadi dasar dalam implementasi program. Alur kerja program merupakan realisasi dari perancangan tersebut, di mana setiap proses dalam program mengikuti aturan DFA yang telah dirancang sebelumnya.

2.4. Implementasi

2.4.1. Alur Program

Program pada Milestone 1 berfungsi sebagai lexical analyzer untuk bahasa Arion. Program menerima input berupa source code Arion dalam format .txt, kemudian membaca karakter input satu per satu menggunakan lexer berbasis Deterministic Finite Automata (DFA). Hasil akhir dari proses ini adalah daftar token yang ditulis ke file output .txt.

Alur program dimulai dari pembacaan argumen command line. Pengguna wajib memberikan path file input, sedangkan path file output dapat diberikan secara eksplisit melalui opsi -o atau --save-tokens. Jika pengguna tidak memberikan path output, program akan menentukan path output default berdasarkan nama file input dan menyimpannya pada folder test/milestone1/output/. Program juga menyediakan opsi --lex-only untuk menjalankan tahap lexical analysis saja, serta opsi --save-dfa-trace untuk menyimpan jejak transisi DFA selama proses tokenisasi berlangsung.

Setelah argumen diproses, program melakukan validasi terhadap file input. Apabila file input tidak ditemukan atau tidak dapat dibuka, program akan menampilkan pesan error melalui std::cerr dan menghentikan eksekusi dengan exit code non-zero. Jika file input valid, program akan memastikan direktori output tersedia. Direktori output dibuat secara otomatis menggunakan std::filesystem apabila belum ada.

Tahap berikutnya adalah inisialisasi DFA. DFA tidak dibangun menggunakan lexer generator atau library eksternal, melainkan dimuat dari file konfigurasi config/config_lexer.txt. File

konfigurasi tersebut berisi definisi start state, final state, finalset untuk keyword, dan aturan transisi antar-state. Dengan pendekatan ini, struktur DFA dapat diperiksa dan disesuaikan tanpa mengubah kode utama, tetapi proses pembacaan token tetap berjalan secara deterministik mengikuti transisi DFA yang telah didefinisikan.

Setelah DFA siap, program membuat objek Lexer dengan input stream, objek DFA, dan output stream token. Lexer kemudian membaca source code karakter demi karakter. Setiap karakter yang dibaca digunakan sebagai input transisi DFA. Selama transisi masih valid, karakter akan digabungkan ke dalam lexeme yang sedang dibentuk. Ketika transisi berikutnya tidak tersedia, lexer memeriksa apakah state terakhir merupakan final state. Jika state terakhir adalah final state, lexeme dikonversi menjadi token. Jika bukan final state, lexeme dianggap sebagai lexical error.

Proses tokenisasi menerapkan prinsip maximal munch atau longest match. Prinsip ini memastikan bahwa lexer selalu mengambil rangkaian karakter terpanjang yang masih valid sebagai sebuah token. Contohnya, karakter `:` yang diikuti `=` akan dikenali sebagai token `becomes`, sedangkan karakter `<` yang diikuti `=` akan dikenali sebagai token `leq`. Dengan demikian, operator multi-karakter tidak terpecah menjadi token yang salah.

Whitespace seperti spasi, tab, dan newline tidak dihasilkan sebagai token karena tidak memiliki makna sintaks pada bahasa Arion. Komentar juga dikenali sebagai token `COMMENT`, tetapi pada tahap berikutnya token komentar dapat diabaikan oleh parser karena tidak memengaruhi struktur program. Lexer mendukung dua bentuk komentar, yaitu komentar dengan delimiter `{ ... }` dan `(* ... *)`.

Untuk identifier dan keyword, lexer terlebih dahulu membentuk lexeme berdasarkan pola identifier. Setelah lexeme terbentuk, DFA melakukan pemetaan finalset untuk menentukan apakah lexeme tersebut merupakan keyword seperti `program`, `var`, `begin`, `end`, `if`, `while`, `for`, dan sebagainya. Jika lexeme tidak termasuk keyword, maka token diklasifikasikan sebagai `ident`.

Apabila lexer menemukan karakter atau rangkaian karakter yang tidak sesuai dengan pola token manapun, lexer menghasilkan token `UNKNOWN` atau melempar `LexerException`. Pada implementasi terbaru, token `UNKNOWN` diperlakukan sebagai lexical error. Program akan menampilkan pesan error yang memuat posisi baris dan kolom, serta menghentikan eksekusi dengan exit code non-zero. Hal ini memastikan input yang tidak valid tidak tetap dianggap berhasil. Program juga dapat menghasilkan DFA trace melalui opsi `--save-dfa-trace`. Trace ini berisi catatan karakter yang dibaca, state asal, state tujuan, dan token yang terbentuk ketika final state dicapai. Fitur ini digunakan sebagai bukti bahwa lexer benar-benar memproses input secara karakter demi karakter mengikuti aturan DFA, sesuai kebutuhan Milestone 1.

Setelah seluruh input selesai dibaca, token-token yang berhasil dikenali ditulis ke file output dalam format satu token per baris. Token yang memiliki nilai lexeme seperti `ident`, `intcon`, `realcon`, `charcon`, `string`, dan `comment` ditampilkan bersama nilainya, sedangkan keyword, operator, dan delimiter ditampilkan berdasarkan nama tokennya. Dengan alur ini, program memenuhi kebutuhan

utama Milestone 1, yaitu membaca source code .txt, memprosesnya dengan DFA, menangani error leksikal secara informatif, dan menghasilkan daftar token yang terformat dengan jelas.

2.4.2. Kode Program Utama

Implementasi

```
#include <fstream>
#include <iostream>
#include <memory>
#include <string>
#include <filesystem>

#include "lexer/lexer.hpp"
#include "lexer/dfa.hpp"
#include "lexer/lexer_exception.hpp"

int main(int argc, char* argv[]) {
    if (argc < 2) {
        std::cerr << "Usage: ./arion <program.txt> [-o
<output_file.txt>]\n";
        return 1;
    }

    std::string input_filename = argv[1];
    std::string output_filename;

    if (argc > 3 && std::string(argv[2]) == "-o") {
        output_filename = argv[3];
    } else {
        std::filesystem::path input_path(input_filename);
        std::string stem = input_path.stem().string();
        output_filename = (std::filesystem::path("test") /
"milestonel" / "output" / (stem + ".txt")).string();
    }

    std::ifstream input_file(input_filename);
    if (!input_file.is_open()) {
        std::cerr << "Gagal membuka file input: " << input_filename
<< "\n";
        return 1;
    }

    std::filesystem::path output_path(output_filename);
    if (output_path.has_parent_path()) {
        std::filesystem::create_directories(output_path.parent_path());
    }
```

```

    std::ofstream output_file(output_filename);
    if (!output_file.is_open()) {
        std::cerr << "Gagal membuka file output: " <<
output_filename << "\n";
        return 1;
    }

    try {
        auto dfa = std::make_shared<DFA>();

        // Ganti path ini jika file konfigurasi DFA Anda berada di
lokasi lain
        dfa->loadConfig("config/config_lexer.txt");

        Lexer lexer(input_file, dfa, &output_file);

        while (!lexer.eof()) {
            lexer.get_next_token();
        }

        std::cout << "Lexing selesai.\n";
        std::cout << "Output disimpan di: " << output_filename <<
"\n";
    }
    catch (const LexerException& e) {
        std::cerr << e.full_message() << "\n";
        return 1;
    }
    catch (const std::exception& e) {
        std::cerr << "ERROR: " << e.what() << "\n";
        return 1;
    }

    return 0;
}

```

Penjelasan

Program ini merupakan *driver* utama untuk melakukan proses analisis leksikal (lexical analysis) terhadap sebuah file input menggunakan pendekatan Deterministic Finite Automaton (DFA). Program menerima nama file input melalui argumen command line, serta opsional nama file output. Jika nama output tidak diberikan, program secara otomatis membuat path output berdasarkan nama file input. Selanjutnya, program membuka file input dan output, serta memastikan direktori tujuan tersedia. Sebuah objek DFA diinisialisasi dan dikonfigurasi dari file eksternal, kemudian digunakan oleh objek *lexer* untuk membaca isi file input karakter demi karakter. Proses lexing dilakukan secara iteratif hingga mencapai akhir file (*end-of-file*), di mana setiap token yang dikenali akan dituliskan ke file output. Program juga dilengkapi dengan mekanisme *exception handling* untuk menangani kesalahan yang mungkin terjadi selama proses, seperti kesalahan pembacaan file atau token yang tidak valid.

Pada implementasi terbaru, program utama juga mendukung beberapa opsi tambahan. Opsi `--lex-only` digunakan untuk menjalankan lexical analysis tanpa melanjutkan ke tahap parser. Opsi `--save-dfa-trace`

digunakan untuk menyimpan jejak transisi DFA ke file terpisah. Selain itu, seluruh error seperti file input tidak ditemukan, lexical error, dan exception umum ditulis melalui `std::cerr` agar output normal dan output error terpisah dengan jelas.

2.4.3. Kode Kelas Token

Header
<pre>#ifndef TOKEN_HPP #define TOKEN_HPP #include <string> class TokenType { private: static int next_id; int id; std::string name; public: TokenType(std::string type_name); int get_type() const; const std::string& get_name() const; }; class Token { private: TokenType type; std::string value; public: Token(TokenType type, std::string value = ""); std::string to_string() const; const std::string& get_value() const; int get_type() const; }; #endif</pre>
Implementasi
<pre>#include "token.hpp" #include <set> int TokenType::next_id = 0; TokenType::TokenType(std::string type_name)</pre>

```

        : id(TokenType::next_id++), name(type_name) {}

int TokenType::get_type() const {
    return id;
}

const std::string& TokenType::get_name() const {
    return name;
}

bool token_has_value(const std::string& type_name) {
    static const std::set<std::string> valued_tokens = {
        "ident", "IDENT",
        "intcon", "INTCON",
        "realcon", "REALCON",
        "charcon", "CHARCON",
        "string", "STRING",
        "unknown", "UNKNOWN",
        "comment", "COMMENT"
    };
    return valued_tokens.find(type_name) != valued_tokens.end();
}

Token::Token(TokenType type, std::string value)
    : type(type), value(value) {}

std::string Token::to_string() const {
    std::string name = type.get_name();
    if (token_has_value(name) && !value.empty()) {
        return name + " (" + value + ")";
    }
    return name;
}

const std::string& Token::get_value() const {
    return value;
}

int Token::get_type() const {
    return type.get_type();
}

```

Penjelasan

Pada bagian awal, variabel statis `TokenType::next_id` diinisialisasi dengan nilai 0 dan digunakan untuk memberikan ID unik secara otomatis pada setiap objek `TokenType` yang dibuat. Konstruktor `TokenType` akan mengisi atribut `id` dengan nilai `next_id` saat ini, lalu menaikannya (increment), sehingga setiap tipe token memiliki identitas numerik yang berbeda. Selain itu, atribut `name` menyimpan nama dari tipe token tersebut. Method `get_type()` dan `get_name()` masing-masing digunakan untuk mengakses ID dan nama tipe token.

Fungsi tambahan `token_has_value()` digunakan untuk menentukan apakah suatu tipe token memiliki nilai (lexeme) yang perlu ditampilkan. Fungsi ini menggunakan struktur data `std::set` untuk menyimpan daftar tipe token seperti identifier, konstanta, string, dan komentar, yang secara umum memang memiliki nilai spesifik dari source code.

Kelas Token diimplementasikan dengan konstruktor yang menerima `TokenType` dan nilai token (opsional). Method `to_string()` berfungsi untuk mengubah token menjadi representasi string. Jika tipe token termasuk dalam kategori yang memiliki nilai (berdasarkan `token_has_value`) dan nilai tidak kosong, maka output akan berbentuk `nama_tipe (nilai)`. Jika tidak, hanya nama tipe yang ditampilkan. Method `get_value()` mengembalikan nilai token, sedangkan `get_type()` mengembalikan ID dari tipe token.

2.4.4. Kode Kelas DFA

Header

```
#pragma once

#include <fstream>
#include <iostream>
#include <cstdint>
#include <vector>
#include <cstring>
#include <sstream>
#include <array>
#include <unordered_map>
#include <string>
#include "token.hpp"

#define N_STATE_CHAR_ID 8
#define MAX_ASCII_USED 128

class State
{
public:
    /**
     * @brief konstruktor default State, menghasilkan null state
     */
    State();
    /**
     * @brief Konstruktor elemen dari state "q0, .. ,q37" yang ada
     pada CONFIG
     * @param charID pointer karakter yang berisikan kode state
     "q0, .. ,q37"
     */
}
```

```

    State(const char *charID, int16_t newStateIdx, bool
isFinalState);
    bool isFinalState() const;
    void setFinalState(bool isFinalState);
    int16_t getStateIdx() const;
    const char *getStateCharID() const;
    bool isNullState() const;
    bool compCharID(const char *otherCharID) const;
    ~State();

private:
    bool finState;
    int16_t stateIdx;
    char stateCharID[N_STATE_CHAR_ID];

    /**
     * @brief fungsi bantu assign untuk assign di State.
     */
    void assignCharID(const char *newCharID);
};

class DFA
{
public:
    DFA();
    /**
     * @brief melakukan load transision termasuk token dari file
konfigurasi
     * @note method ini menjadi satu-satunya cara untuk mengisi
atribut kelas DFA persis setelah inisiasi.
     * @note START harus selalu di baris pertama file.
     */
    void loadConfig(std::string path);

    /**
     * @brief metode yang dibuat dengan tujuan debugging,
mem-visualisasikan proses perpindahan state dan scanner pada DFA.
     * @note hasil dari visualisasi disimpan pada path input.
     */
    void visualizeProcess(std::string path) const;
    /**
     * @brief export DFA config for debugging purpose.
     */
    void exportDFAConfig(std::string path) const;

    const State &getState() const;

    /**
     * @brief next tidak menghasilkan karakter karena DFA hanya
bertanggung jawab terhadap perubahasn state saja.
     * @note penyimpanan value dilakukan di luar DFA.

```

```

    */
    void next(unsigned char c);
    /**
     * @brief fungsi untuk melakukan reset state sehingga current
    state kembali ke start state
     */
    void resetState();
    TokenType getCurrToken() const;
    TokenType getTokenForState(int stateID) const;
    int getTokIDfromStateID(int stateID) const;
    int getTokIDfromTokName(std::string tokName) const;
    State getStateWithId(int stateId) const;
    bool hasKeywordToken(const std::string &lexeme) const;
    TokenType getKeywordToken(const std::string &lexeme) const;

    ~DFA();

private:
    int16_t currStateIdx;
    static State nullState;
    static TokenType unknownToken;

    static bool isVisualized;
    static std::fstream visualFileStream;

    std::vector<State> states;
    std::vector<std::array<int16_t, MAX_ASCII_USED> > transTable;

    // Penyimpanan TokenType
    /**
     * @brief tempat menyimpan kumpulan dari TokenType
     * @note id dari TokenType sama dengna index pada vector
    tokenTypes
     */
    std::vector<TokenType> tokenTypes;

    /**
     * @brief tokenNameIDMapping memetakan nama ke id dari
    TokenType
     * @note tujuannya adalah untuk mencegah pembuatan token yang
    sama dua kali.
     */
    std::unordered_map<std::string, int> tokenNameIDMapping;

    /**
     * @brief stateIDtoTokenID merupakan pemetaan antara stateID ke
    Token ID
     * @note tujuannya adalah supaya setiap (final) state yang
    terhubung dengan token ID dapat mengakses TokenTypes secara tidak
    langsung melalui StateID-nya
     */

```



```

std::unordered_map<int, int> stateIDtoTokenID;
std::unordered_map<std::string, int> keywordLexemeToTokenID;

int addUniqueState(const char *newCharID, bool newFinState);
int findStateIdx(const char *newCharID);
void setCurrentState(int16_t newStateIdx);
void addTransition(int16_t state1, int input, int16_t state2);

void addTokenToTokenTypes(TokenType newTok);
void addToTokenNameIDMapping(std::string name, int tokID);
void addToStateIDtoTokenID(int stateID, int tokID);
void addKeywordLexemeToTokenID(const std::string &lexeme, int
tokID);
int addOrGetTokenID(const std::string &name);
static std::string toUpper(std::string text);

void visualizedProccToFile(char c, State currState) const;
};

```

Implementasi

```

#include "dfa.hpp"

// Implementasi State
State::State() : stateIdx(-1), finState(false) {
    assignCharID("null");
}

State::State(const char *id, int16_t idx, bool isFinal)
    : stateIdx(idx), finState(isFinal) {
    assignCharID(id);
}

bool State::isFinalState() const { return finState; }
int16_t State::getStateIdx() const { return stateIdx; }

void State::assignCharID(const char *id) {
    strncpy(stateCharID, id, N_STATE_CHAR_ID - 1);
    stateCharID[N_STATE_CHAR_ID - 1] = '\0';
}

// Implementasi DFA
DFA::DFA() : currStateIdx(-1) {}

void DFA::resetState() {
    currStateIdx = 0; // start state
}

void DFA::next(unsigned char c) {

```

```

        currStateIdx = transTable[currStateIdx][c];
    }

    const State& DFA::getState() const {
        if (currStateIdx < 0 || currStateIdx >= states.size())
            return nullState;
        return states[currStateIdx];
    }

    // Load Config
    void DFA::loadConfig(std::string path) {
        std::fstream fs(path);
        std::string line;

        while (getline(fs, line)) {
            std::istringstream s(line);
            std::string cmd;
            s >> cmd;

            if (cmd == "START") {
                std::string st;
                s >> st;
                addUniqueState(st.c_str(), false);
                resetState();
            }
            else if (cmd == "FINAL") {
                std::string tok, st;
                s >> tok >> st;
                int tokId = addOrGetTokenID(tok);
                int stId = addUniqueState(st.c_str(), true);
                addToStateIDtoTokenID(stId, tokId);
            }
            else {
                int ascii = stoi(cmd);
                std::string from, to;
                s >> from >> to;
                int f = addUniqueState(from.c_str(), false);
                int t = addUniqueState(to.c_str(), false);
                addTransition(f, ascii, t);
            }
        }
    }

    // Transisi
    void DFA::addTransition(int16_t s1, int input, int16_t s2) {
        while (transTable.size() <= s1) {
            std::array<int16_t, MAX_ASCII_USED> row;
            row.fill(-1);
            transTable.push_back(row);
        }
        transTable[s1][input] = s2;
    }

```

```
}
```

Penjelasan

Kelas State merepresentasikan setiap state dalam automata, yang memiliki identitas (ID), nama state, serta penanda apakah state tersebut merupakan final state. State digunakan untuk menentukan apakah suatu rangkaian karakter sudah membentuk token yang valid.

Kelas DFA berfungsi sebagai mesin utama yang mengatur perpindahan antar state berdasarkan input karakter. Transisi antar state disimpan dalam bentuk tabel (transition table) yang memetakan state saat ini dan input karakter ke state berikutnya. Method next() digunakan untuk berpindah state sesuai karakter yang dibaca, sedangkan resetState() mengembalikan DFA ke state awal.

Konfigurasi DFA dibaca dari file eksternal melalui method loadConfig, yang mendefinisikan state awal, state akhir (final), serta aturan transisi. Dengan pendekatan ini, struktur DFA menjadi fleksibel dan dapat diubah tanpa memodifikasi kode program. Selain itu, DFA juga mengelola pemetaan antara state akhir dan token yang dihasilkan, sehingga setiap pola input yang dikenali dapat dikonversi menjadi token yang sesuai.

2.4.5. Kode Kelas Lexer

Header

```
// Header Lexer keseluruhan
#ifndef LEXER_HPP
#define LEXER_HPP

#include <istream>
#include <ostream>
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>

#include "dfa.hpp"
#include "token.hpp"
#include "lexer_exception.hpp"

class Lexer {
public:
    Lexer(std::istream& source, std::shared_ptr<DFA> automaton,
std::ostream* output = nullptr);
    bool eof() const;
    void process_next_token();
    const std::vector<Token>& getResult() const;

    bool hasErrors() const;
    const std::vector<std::string>& getErrors() const;
```

```

        void enableTrace(std::ostream* traceOutput);

private:
    std::istream& src;
    std::shared_ptr<DFA> dfa;
    std::ostream* out;
    std::ostream* traceOut;

    int line_counter;
    int col_counter;
    bool reached_eof;

    bool read_char(char& c);
    void update_position(char c);
    void write_token(const Token& t);
    void write_range_tokens(const std::string& lexeme);
    void add_error(const std::string& message, int line, int col,
const std::string& lexeme);
    void trace_transition(char c, const State& from, const State&
to, const std::string& currentLexeme) const;

    std::vector<Token> result;
    std::vector<std::string> errors;
};

#endif

```

Implementasi

```

#include "lexer.hpp"

#include <cctype>
#include <iostream>
#include <optional>

Lexer::Lexer(std::istream& source, std::shared_ptr<DFA> automaton,
std::ostream* output)
    : src(source),
      dfa(automaton),
      out(output),
      traceOut(nullptr),
      line_counter(1),
      col_counter(0),
      reached_eof(false)
{}

bool Lexer::eof() const {
    return reached_eof;
}

bool Lexer::read_char(char& c) {

```

```

        if (!src.get(c)) {
            reached_eof = true;
            return false;
        }
        return true;
    }

void Lexer::update_position(char c) {
    if (c == '\n') {
        ++line_counter;
        col_counter = 0;
    } else {
        ++col_counter;
    }
}

static bool isWhitespace(char c) {
    return c == ' ' || c == '\t' || c == '\r' || c == '\n';
}

static bool isTokenBoundary(char c) {
    switch (c) {
        case ' ': case '\t': case '\r': case '\n':
        case '+': case '-': case '*': case '/':
        case '<': case '>': case '=': case ':':
        case '.': case ',': case ';':
        case '(': case ')': case '[': case ']':
        case '{': case '}': case '\\':
            return true;
        default:
            return false;
    }
}

static bool isValueLikeToken(const std::string& name) {
    return name == "IDENT" || name == "INTCON" ||
        name == "REALCON" || name == "RANGE";
}

static std::string decodeQuotedValue(const std::string& raw) {
    if (raw.size() < 2) return raw;
    std::string content = raw.substr(1, raw.size() - 2);
    std::string decoded;
    for (size_t i = 0; i < content.size(); ++i) {
        if (content[i] == '\\' && i + 1 < content.size() &&
content[i + 1] == '\\') {
            decoded += '\\';
            ++i;
        } else {
            decoded += content[i];
        }
    }
}

```

```

    }
    return "\"" + decoded + "\"";
}

void Lexer::write_range_tokens(const std::string& lexeme) {
    size_t dotdot = lexeme.find("..");
    std::string first = lexeme.substr(0, dotdot);
    std::string second = lexeme.substr(dotdot + 2);
    TokenType intcon = dfa->getTokenFromTypeName("INTCON");
    TokenType dot = dfa->getTokenFromTypeName("PERIOD");
    write_token(Token(intcon, first));
    write_token(Token(dot, "."));
    write_token(Token(dot, "."));
    write_token(Token(intcon, second));
}

void Lexer::write_token(const Token& t) {
    result.push_back(t);
    if (t.get_type_name() == "UNKNOWN") {
        add_error("token tidak valid", line_counter, col_counter,
t.get_value());
    }
    if (out != nullptr) {
        *out << t.to_string() << "\n";
    }
}

bool Lexer::hasErrors() const {
    return !errors.empty();
}

const std::vector<std::string>& Lexer::getErrors() const {
    return errors;
}

void Lexer::enableTrace(std::ostream* traceOutput) {
    traceOut = traceOutput;
}

static std::string printableChar(char c) {
    switch (c) {
        case '\n': return "\\n";
        case '\t': return "\\t";
        case '\r': return "\\r";
        default: return std::string(1, c);
    }
}

void Lexer::add_error(const std::string& message, int line, int
col, const std::string& lexeme) {
    std::string full = "[" + std::to_string(line) + ":" +

```

```

std::to_string(col) + "] lexical error: " + message;
    if (!lexeme.empty()) {
        full += " (near '" + lexeme + " ')" ;
    }
    errors.push_back(full);
}

void Lexer::trace_transition(char c, const State& from, const
State& to, const std::string& currentLexeme) const {
    if (traceOut == nullptr) {
        return;
    }

    *traceOut << "[" << line_counter << ":" << col_counter << "]" "
        << "read='" << printableChar(c) << "' "
        << from.getStateCharID() << " -> " <<
to.getStateCharID()
        << " lexeme=\"" << currentLexeme << printableChar(c)
<< "\"";

    if (to.isFinalState()) {
        *traceOut << " ACCEPT(" <<
dfa->getTokenForState(to.getStateIdx()).get_name() << ")";
    }

    if (to.isNullState()) {
        *traceOut << " REJECT";
    }

    *traceOut << "\n";
}

void Lexer::process_next_token() {
    if (reached_eof) {
        return;
    }

    TokenType unknownType =
dfa->getTokenTypeFromTypeName("UNKNOWN");
    TokenType commentType =
dfa->getTokenTypeFromTypeName("COMMENT");
    TokenType identType = dfa->getTokenTypeFromTypeName("IDENT");

    auto make_token = [&](TokenType tt, const std::string& lex) ->
Token {
        const std::string& name = tt.get_name();
        if (name == "CHARCON" || name == "STRING") {
            return Token(tt, decodeQuotedValue(lex));
        }
        return Token(tt, lex);
    };
};

```

```

auto emit_unknown = [&](const std::string& lex) {
    if (!lex.empty()) {
        write_token(Token(unknownType, lex));
    }
};

auto report_lexical_error = [&](const std::string& message,
                                int line,
                                int col,
                                const std::string& lex) {
    add_error(message, line, col, lex);
};

auto report_unclosed_quote = [&](int line, int col, const
std::string& lex) {
    report_lexical_error("string/char literal belum ditutup",
line, col, lex);
};

auto emit_accepted = [&](TokenType tt, const std::string& lex)
{
    if (lex.empty()) {
        return;
    }
    if (tt.get_name() == "RANGE") {
        write_range_tokens(lex);
        return;
    }
    write_token(make_token(tt, lex));
};

std::optional<char> pending;

auto scan_comment = [&](std::string lexeme, char prev, int
start_line, int start_col) {
    char c;
    while (read_char(c)) {
        update_position(c);
        lexeme += c;
        if (c == '}' || (prev == '*' && c == ' ')) {
            write_token(Token(commentType, lexeme));
            return;
        }
        prev = c;
    }
    report_lexical_error("unterminated comment", start_line,
start_col, lexeme);
    emit_unknown(lexeme);
};

```



```

auto scan_unknown_tail = [&](std::string lexeme) {
    char c;
    while (read_char(c)) {
        update_position(c);
        if (isWhitespace(c)) {
            emit_unknown(lexeme);
            return;
        }
        if (isTokenBoundary(c)) {
            emit_unknown(lexeme);
            pending = c;
            return;
        }
        lexeme += c;
    }
    emit_unknown(lexeme);
};

auto scan_identifier_tail = [&](std::string lexeme) {
    char c;
    while (read_char(c)) {
        update_position(c);
        if (std::isalnum(static_cast<unsigned char>(c))) {
            lexeme += c;
            continue;
        }
        pending = c;
        break;
    }
    write_token(Token(identType, lexeme));
};

dfa->resetState();
std::string lexeme;
int token_start_line = line_counter;
int token_start_col = col_counter;
char c;

while (pending.has_value() || read_char(c)) {
    if (pending.has_value()) {
        c = pending.value();
        pending = std::nullopt;
    } else {
        update_position(c);
    }

    bool consume_current = true;
    while (consume_current) {
        consume_current = false;

```

```

        if (lexeme.empty() && isWhitespace(c)) {
            dfa->resetState();
            break;
        }

        if (lexeme.empty()) {
            token_start_line = line_counter;
            token_start_col = col_counter;
        }

        if (lexeme.empty() && c == '{') {
            dfa->resetState();
            scan_comment("{", '{', token_start_line,
token_start_col);
            break;
        }

        const State prev_state = dfa->getState();
        dfa->next(static_cast<unsigned char>(c));
        const State next_state = dfa->getState();
        trace_transition(c, prev_state, next_state, lexeme);

        if (!next_state.isNullState()) {
            TokenType next_token =
dfa->getTokenForState(next_state.getStateIdx());
            if (next_state.isFinalState() &&
next_token.get_name() == "UNKNOWN" &&
                !lexeme.empty() &&
                std::isalpha(static_cast<unsigned
char>(lexeme.front())) &&
                std::isalnum(static_cast<unsigned char>(c))) {
                lexeme += c;
                dfa->resetState();
                scan_identifier_tail(lexeme);
                lexeme.clear();
                break;
            }

            lexeme += c;

            if (lexeme == "(*") {
                dfa->resetState();
                scan_comment(lexeme, '*', token_start_line,
token_start_col);
                lexeme.clear();
            }
            break;
        }

        TokenType prev_token =
dfa->getTokenForState(prev_state.getStateIdx());

```

```

        if (prev_state.isFinalState() && prev_token.get_name()
!= "UNKNOWN") {
            if (prev_token.get_name() == "IDENT" &&
std::isalnum(static_cast<unsigned char>(c))) {
                lexeme += c;
                dfa->resetState();
                scan_identifier_tail(lexeme);
                lexeme.clear();
                break;
            }

            if (isValueLikeToken(prev_token.get_name()) &&
!isTokenBoundary(c)) {
                emit_accepted(prev_token, lexeme);
                lexeme.clear();
                dfa->resetState();
                consume_current = true;
                continue;
            }

            emit_accepted(prev_token, lexeme);
            lexeme.clear();
            dfa->resetState();

            if (!isWhitespace(c)) {
                consume_current = true;
                continue;
            }
            break;
        }

        if (lexeme.empty()) {
            if (!isWhitespace(c)) {
                emit_unknown(std::string(1, c));
            }
            dfa->resetState();
            break;
        }

        if (isWhitespace(c)) {
            if (!lexeme.empty() && lexeme.front() == '\\') {
                report_unclosed_quote(token_start_line,
token_start_col, lexeme);
            }
            emit_unknown(lexeme);
            lexeme.clear();
            dfa->resetState();
            break;
        }

        if (isTokenBoundary(c)) {

```

```

        emit_unknown(lexeme);
        lexeme.clear();
        dfa->resetState();
        consume_current = true;
        continue;
    }

    if (!lexeme.empty() &&
        std::isalpha(static_cast<unsigned
char>(lexeme.front())) &&
        std::isalnum(static_cast<unsigned char>(c))) {
        lexeme += c;
        dfa->resetState();
        scan_identifier_tail(lexeme);
        lexeme.clear();
        break;
    }

    lexeme += c;
    dfa->resetState();
    scan_unknown_tail(lexeme);
    lexeme.clear();
    break;
}

const State cur = dfa->getState();
if (!lexeme.empty()) {
    if (cur.isFinalState()) {
        emit_accepted(dfa->getCurrToken(), lexeme);
    } else {
        if (lexeme.front() == '\\') {
            report_unclosed_quote(token_start_line,
token_start_col, lexeme);
        }
        emit_unknown(lexeme);
    }
}

reached_eof = true;
}

const std::vector<Token>& Lexer::getResult() const {
    return result;
}

```

Penjelasan

Kode ini merupakan implementasi dari kelas Lexer yang bertugas melakukan proses analisis leksikal, yaitu membaca input karakter dan mengubahnya menjadi token menggunakan DFA. Lexer membaca

input secara bertahap menggunakan fungsi `read_char`, lalu memperbarui posisi baris dan kolom untuk keperluan pelacakan error.

Proses utama terjadi pada fungsi `get_next_token`, di mana lexer akan mengumpulkan karakter menjadi sebuah lexeme dan mengirimkannya ke DFA untuk menentukan perpindahan state. Jika DFA mencapai final state, maka lexeme dianggap sebagai token yang valid dan akan dikembalikan serta dituliskan ke output. Jika terjadi transisi ke state tidak valid (null state) sebelum mencapai final state, maka lexer akan menghasilkan error. Lexer juga menangani pengabaian whitespace di antara token, serta kondisi end-of-file (EOF) untuk memastikan token terakhir tetap diproses dengan benar.

2.4.6. Config

Implementasi
<pre># Define q0 as start state START q0 # Final State Format # FINAL [TOKEN_TYPE] [State]* #Finalset Format # FINALSET [TOKEN_TYPE] [State] [value]* # Transition Format # [Input State] [Input] [New State]</pre>
Penjelasan
File <code>config/config_lexer.txt</code> digunakan sebagai sumber konfigurasi DFA. Konfigurasi ini berisi beberapa jenis baris utama, yaitu: <pre>START <state> FINAL <token_type> <state> FINALSET <token_type> <state> <lexeme> <ascii_code> <from_state> <to_state></pre> START menentukan state awal DFA. FINAL menentukan state akhir yang langsung dipetakan ke tipe token tertentu. FINALSET digunakan untuk membedakan keyword dari identifier berdasarkan lexeme yang terbentuk. Sementara itu, baris transisi menggunakan kode ASCII untuk menunjukkan perpindahan dari satu state ke state lain berdasarkan karakter input. Dengan format konfigurasi tersebut, DFA tetap bersifat eksplisit dan dapat diperiksa, tetapi implementasi program tidak perlu menuliskan seluruh transisi dalam bentuk percabangan panjang di source code. Hal ini juga memudahkan sinkronisasi antara diagram DFA dan implementasi transisi yang dipakai program. • Start State : digunakan untuk mendefinisikan state awal dalam DFA. State ini menjadi titik awal proses pembacaan input. • Final state : digunakan untuk menandai bahwa suatu state merupakan state akhir (<i>accepting</i>)

state). State ini menunjukkan bahwa input yang diproses telah dikenali sebagai token yang valid dengan tipe tertentu.

- Finalset State : digunakan untuk memetakan string yang diterima pada suatu state tertentu ke dalam nilai spesifik dari tipe token yang berbeda. Dengan demikian, state yang sama dapat menghasilkan nilai token yang berbeda berdasarkan string yang dikenali.
- Transition Format : digunakan untuk mendefinisikan perpindahan antar state berdasarkan input tertentu. Setiap aturan transisi menunjukkan bagaimana sistem berpindah dari satu state ke state lain ketika menerima suatu simbol input.

BAB III

PENGUJIAN

3.1. Pengujian Pertama

Input	Output
<pre>input1.txt x MAR-Tubes-IF2224-2026 > test > milestone1 > input > input1.txt 1 program Hello; 2 var 3 a, b: integer; 4 begin 5 a := 5; 6 b := a + 10; 7 writeln('Result = ', b); 8 end.</pre>	<pre>test > milestone1 > output > output1 1 PROGRAMSY 2 IDENT (Hello) 3 SEMICOLON 4 VARSY 5 IDENT (a) 6 COMMA 7 IDENT (b) 8 COLON 9 IDENT (integer) 10 SEMICOLON 11 BEGINSY 12 IDENT (a) 13 BECOMES 14 INTCON (5) 15 SEMICOLON 16 IDENT (b) 17 BECOMES 18 IDENT (a) 19 PLUS 20 INTCON (10) 21 SEMICOLON 22 IDENT (writeln) 23 LPARENT 24 STRING ('Result = ') 25 COMMA 26 IDENT (b) 27 RPARENT 28 SEMICOLON 29 ENDSY 30 PERIOD 31</pre>
Keterangan	
Pengujian ini dilakukan menggunakan program Arion sederhana yang valid secara sintaksis, sesuai dengan contoh yang terdapat pada spesifikasi tugas. Tujuan pengujian ini adalah untuk memastikan bahwa lexer mampu mengenali token-token dengan benar, seperti keyword, identifier, operator, konstanta integer, string, serta simbol-simbol seperti tanda kurung, koma,	

titik koma, titik, dan titik dua.

3.2. Pengujian Kedua

Input	Output
<pre>≡ input2.txt × MAR-Tubes-IF2224-2026 > test > milestone1 > input > ≡ input2.txt 1 PROGRAM CaseTest; 2 VAR 3 x: integer; 4 BEGIN 5 x := 42; 6 IF x > 10 THEN 7 writeln('besar') 8 ELSE 9 writeln('kecil'); 10 END.</pre>	<pre>test > milestone1 > output > ≡ output2 1 PROGRAMSY 2 IDENT (CaseTest) 3 SEMICOLON 4 VARSY 5 IDENT (x) 6 COLON 7 IDENT (integer) 8 SEMICOLON 9 BEGINSY 10 IDENT (x) 11 BECOMES 12 INTCON (42) 13 SEMICOLON 14 IFSY 15 IDENT (x) 16 GTR 17 INTCON (10) 18 THENSY 19 IDENT (writeln) 20 LPARENT 21 STRING ('besar') 22 RPARENT 23 ELSESY 24 IDENT (writeln) 25 LPARENT 26 STRING ('kecil') 27 RPARENT 28 SEMICOLON 29 ENDSY 30 PERIOD 31</pre>
Keterangan	
Pengujian ini dilakukan menggunakan program Arion sederhana yang memanfaatkan penulisan keyword dalam huruf besar, seperti PROGRAM, VAR, BEGIN, IF, THEN, ELSE, dan END. Tujuan pengujian ini adalah untuk memastikan bahwa lexer tetap dapat mengenali keyword	

secara benar meskipun penulisannya menggunakan huruf besar, serta tetap mampu mengenali identifier, konstanta integer, operator perbandingan, string, tanda kurung, titik koma, dan token lainnya dengan tepat.

3.3. Pengujian Ketiga

Input	Output
-------	--------

≡ input3.txt X

MAR-Tubes-IF2224-2026 > test > milestone1 > input > ≡ input3.txt

```
1  program RelasiTest;
2  var
3  a, b: integer;
4  begin
5  a := 10;
6  b := 20;
7  if a == b then writeln('sama');
8  if a <> b then writeln('beda');
9  if a < b then writeln('kurang');
10 if a <= b then writeln('kurang sama');
11 if a > b then writeln('lebih');
12 if a >= b then writeln('lebih sama');
13 end.
```

test > milestone1 > output > ≡ output3

```
1  PROGRAMSY
2  IDENT (RelasiTest)
3  SEMICOLON
4  VARSY
5  IDENT (a)
6  COMMA
7  IDENT (b)
8  COLON
9  IDENT (integer)
10 SEMICOLON
11 BEGINSY
12 IDENT (a)
13 BECOMES
14 INTCON (10)
15 SEMICOLON
16 IDENT (b)
17 BECOMES
18 INTCON (20)
19 SEMICOLON
20 IFSY
21 IDENT (a)
22 EQL
23 IDENT (b)
24 THENSY
25 IDENT (writeln)
26 LPARENT
27 STRING ('sama')
28 RPARENT
29 SEMICOLON
30 IFSY
31 IDENT (a)
32 NEQ
33 IDENT (b)
34 THENSY
35 IDENT (writeln)
36 LPARENT
37 STRING ('beda')
38 RPARENT
39 SEMICOLON
40 IFSY
41 IDENT (a)
```

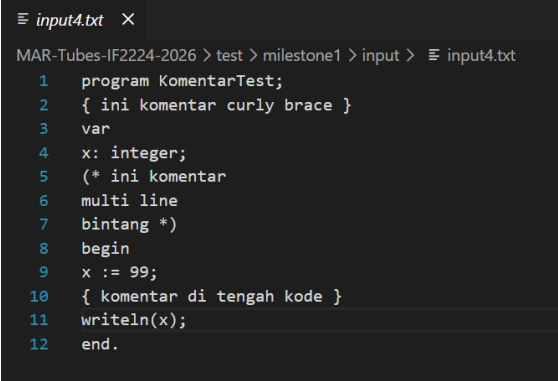
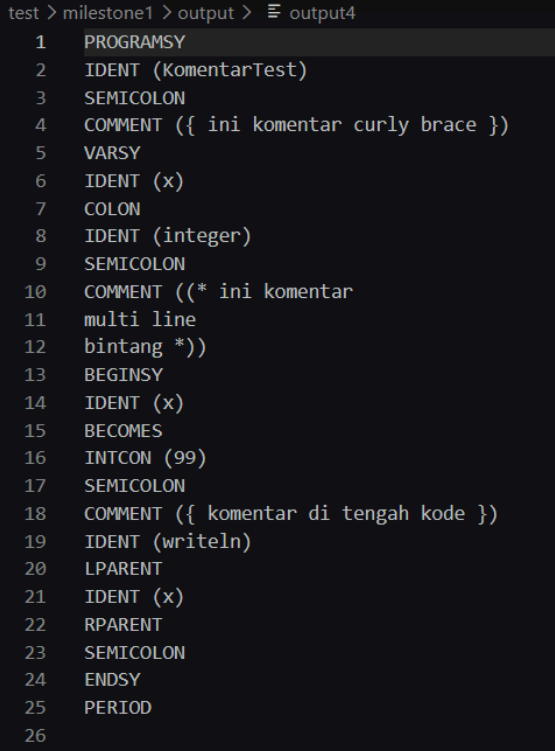
test > milestone1 > output >  output3

```
42  LSS
43  IDENT (b)
44  THENSY
45  IDENT (writeln)
46  LPARENT
47  STRING ('kurang')
48  RPARENT
49  SEMICOLON
50  IFSY
51  IDENT (a)
52  LEQ
53  IDENT (b)
54  THENSY
55  IDENT (writeln)
56  LPARENT
57  STRING ('kurang sama')
58  RPARENT
59  SEMICOLON
60  IFSY
61  IDENT (a)
62  GTR
63  IDENT (b)
64  THENSY
65  IDENT (writeln)
66  LPARENT
67  STRING ('lebih')
68  RPARENT
69  SEMICOLON
70  IFSY
71  IDENT (a)
72  GEQ
73  IDENT (b)
74  THENSY
75  IDENT (writeln)
76  LPARENT
77  STRING ('lebih sama')
78  RPARENT
79  SEMICOLON
80  ENDSY
81  PERIOD
```

Keterangan

Pengujian ini dilakukan menggunakan program Arion sederhana yang memuat berbagai operator relasi, yaitu ==, <>, <, <=, >, dan >=. Tujuan pengujian ini adalah untuk memastikan bahwa lexer mampu mengenali seluruh operator relasi dengan benar, sekaligus tetap mengenali keyword, identifier, konstanta integer, string, tanda kurung, dan titik koma sesuai token yang semestinya.

3.4. Pengujian Keempat

Input	Output
 <pre> MAR-Tubes-IF2224-2026 > test > milestone1 > input > ≡ input4.txt 1 program KomentarTest; 2 { ini komentar curly brace } 3 var 4 x: integer; 5 (* ini komentar 6 multi line 7 bintang *) 8 begin 9 x := 99; 10 { komentar di tengah kode } 11 writeln(x); 12 end. </pre>	 <pre> test > milestone1 > output > ≡ output4 1 PROGRAMSY 2 IDENT (KomentarTest) 3 SEMICOLON 4 COMMENT ({ ini komentar curly brace }) 5 VARSY 6 IDENT (x) 7 COLON 8 IDENT (integer) 9 SEMICOLON 10 COMMENT ((* ini komentar 11 multi line 12 bintang *)) 13 BEGINSY 14 IDENT (x) 15 BECOMES 16 INTCON (99) 17 SEMICOLON 18 COMMENT ({ komentar di tengah kode }) 19 IDENT (writeln) 20 LPARENT 21 IDENT (x) 22 RPARENT 23 SEMICOLON 24 ENDSY 25 PERIOD 26 </pre>
Keterangan	
Pengujian ini dilakukan menggunakan program Arion yang memuat komentar dengan dua bentuk penulisan, yaitu komentar yang diawali dan diakhiri dengan tanda kurung kurawal ({ ... }) serta komentar yang diawali dengan (*) dan diakhiri dengan (*). Tujuan pengujian ini adalah untuk memastikan bahwa lexer mampu mengenali token comment dengan benar, baik untuk komentar satu baris maupun komentar multibaris, tanpa mengganggu proses pembacaan token-token lain pada program.	

3.5. Pengujian Kelima

Input	Output
-------	--------

MAR-Tubes-IF2224-2026 > test > milestone1 > input >  input5.txt

```
1  program LogikaTest;
2  var
3  a, b, c: integer;
4  begin
5  a := 10;
6  b := 5;
7  c := a div b;
8  c := a mod b;
9  c := a * b;
10 c := a / b;
11 if (a > 0) AND (b > 0) then writeln('positif');
12 if (a > 0) OR (b < 0) then writeln('salah satu positif');
13 if NOT (a == b) then writeln('tidak sama');
14 end.
```

test > milestone1 > output >  output5

```
1  PROGRAMSY
2  IDENT (LogikaTest)
3  SEMICOLON
4  VARSY
5  IDENT (a)
6  COMMA
7  IDENT (b)
8  COMMA
9  IDENT (c)
10 COLON
11 IDENT (integer)
12 SEMICOLON
13 BEGINSY
14 IDENT (a)
15 BECOMES
16 INTCON (10)
17 SEMICOLON
18 IDENT (b)
19 BECOMES
20 INTCON (5)
21 SEMICOLON
22 IDENT (c)
23 BECOMES
24 IDENT (a)
25 IDIV
26 IDENT (b)
27 SEMICOLON
28 IDENT (c)
29 BECOMES
30 IDENT (a)
31 IMOD
32 IDENT (b)
33 SEMICOLON
34 IDENT (c)
35 BECOMES
36 IDENT (a)
37 TIMES
38 IDENT (b)
39 SEMICOLON
40 IDENT (c)
41 BECOMES
42 IDENT (a)
43 RDIV
44 IDENT (b)
45 SEMICOLON
46 IFSY
47 LPARENT
48 IDENT (a)
49 GTR
50 INTCON (0)
```

test > milestone1 > output >  output5

```
51  RPARENT
52  ANDSY
53  LPARENT
54  IDENT (b)
55  GTR
56  INTCON (0)
57  RPARENT
58  THENSY
59  IDENT (writeln)
60  LPARENT
61  STRING ('positif')
62  RPARENT
63  SEMICOLON
64  IFSY
65  LPARENT
66  IDENT (a)
67  GTR
68  INTCON (0)
69  RPARENT
70  ORSY
71  LPARENT
72  IDENT (b)
73  LSS
74  INTCON (0)
75  RPARENT
76  THENSY
77  IDENT (writeln)
78  LPARENT
79  STRING ('salah satu positif')
80  RPARENT
81  SEMICOLON
82  IFSY
83  NOTSY
84  LPARENT
85  IDENT (a)
86  EQL
87  IDENT (b)
88  RPARENT
89  THENSY
90  IDENT (writeln)
91  LPARENT
92  STRING ('tidak sama')
93  RPARENT
94  SEMICOLON
95  ENDSY
96  PERIOD
97
```

Keterangan

Pengujian ini dilakukan menggunakan program Arion yang memuat operator aritmatika dan operator logika, yaitu div, MOD, *, /, AND, OR, dan NOT. Tujuan pengujian ini adalah untuk memastikan bahwa lexer mampu mengenali token-token tersebut dengan benar, sekaligus tetap mengenali keyword, identifier, konstanta integer, operator relasi, string, tanda kurung, dan titik koma sesuai dengan token yang semestinya.

3.6. Pengujian Keenam

Input	Output
-------	--------

test > milestone1 > input > input6.txt

```
1  program LoopTest;
2  var
3  n: integer;
4  begin
5  n := 1;
6  while n <= 5 do
7  begin
8  writeln(n);
9  n := n + 1;
10 end;
11 repeat
12 n := n - 1;
13 until n == 0;
14 case n of
15 0: writeln('no!');
16 end;
17 end.
```

```
1  PROGRAMSY
2  IDENT (LoopTest)
3  SEMICOLON
4  VARSY
5  IDENT (n)
6  COLON
7  IDENT (integer)
8  SEMICOLON
9  BEGINSY
10 IDENT (n)
11 BECOMES
12 INTCON (1)
13 SEMICOLON
14 WHILESY
15 IDENT (n)
16 LEQ
17 INTCON (5)
18 DOSY
19 BEGINSY
20 IDENT (writeln)
21 LPARENT
22 IDENT (n)
23 RPARENT
24 SEMICOLON
25 IDENT (n)
26 BECOMES
27 IDENT (n)
28 PLUS
29 INTCON (1)
30 SEMICOLON
31 ENDSY
32 SEMICOLON
33 REPEATSY
34 IDENT (n)
35 BECOMES
36 IDENT (n)
37 MINUS
38 INTCON (1)
39 SEMICOLON
40 UNTILSY
41 IDENT (n)
42 EQL
43 INTCON (0)
44 SEMICOLON
45 CASESY
46 IDENT (n)
47 OFSY
48 INTCON (0)
49 COLON
```


	<pre> 35 BECOMES 36 IDENT (n) 37 MINUS 38 INTCON (1) 39 SEMICOLON 40 UNTILSY 41 IDENT (n) 42 EQL 43 INTCON (0) 44 SEMICOLON 45 CASESY 46 IDENT (n) 47 OFSY 48 INTCON (0) 49 COLON 50 IDENT (writeln) 51 LPARENT 52 STRING ('no1') 53 RPARENT 54 SEMICOLON 55 ENDSY 56 SEMICOLON 57 ENDSY 58 PERIOD 59 </pre>
Keterangan	
Pengujian ini dilakukan menggunakan program Arion yang memuat beberapa bentuk perulangan dan percabangan, yaitu while ... do, repeat ... until, dan case ... of. Tujuan pengujian ini adalah untuk memastikan bahwa lexer mampu mengenali keyword yang berkaitan dengan struktur kontrol program tersebut dengan benar, sekaligus tetap mengenali identifier, konstanta integer, operator relasi, operator aritmatika, string, tanda kurung, titik dua, titik koma, dan titik sesuai token yang semestinya.	

3.7. Pengujian Ketujuh

Input	Output
--------------	---------------

```
test > milestone1 > input > ≡ input7.txt
```

```
1  program TipeData;  
2  var  
3  r: real;  
4  c: char;  
5  begin  
6  r := 3.14;  
7  r := 26.7;  
8  c := 'A';  
9  c := 'z';  
10 writeln(r);  
11 writeln(c);  
12 end.
```

```
1  PROGRAMSY  
2  IDENT (TipeData)  
3  SEMICOLON  
4  VARSY  
5  IDENT (r)  
6  COLON  
7  IDENT (real)  
8  SEMICOLON  
9  IDENT (c)  
10 COLON  
11 IDENT (char)  
12 SEMICOLON  
13 BEGINSY  
14 IDENT (r)  
15 BECOMES  
16 REALCON (3.14)  
17 SEMICOLON  
18 IDENT (r)  
19 BECOMES  
20 REALCON (26.7)  
21 SEMICOLON  
22 IDENT (c)  
23 BECOMES  
24 CHARCON ('A')  
25 SEMICOLON  
26 IDENT (c)  
27 BECOMES  
28 CHARCON ('z')  
29 SEMICOLON  
30 IDENT (writeln)  
31 LPARENT  
32 IDENT (r)  
33 RPARENT  
34 SEMICOLON  
35 IDENT (writeln)  
36 LPARENT  
37 IDENT (c)  
38 RPARENT  
39 SEMICOLON  
40 ENDSY  
41 PERIOD  
42
```

Keterangan

Pengujian ini dilakukan menggunakan program Arion yang memuat konstanta bilangan riil dan konstanta karakter. Tujuan pengujian ini adalah untuk memastikan bahwa lexer mampu mengenali token realcon dan charcon dengan benar, serta tetap dapat mengenali keyword,

identifier, operator assignment, tanda kurung, titik koma, dan titik sesuai dengan token yang semestinya.

3.8. Pengujian Kedelapan

Input	Output
<pre>test > milestone1 > input > ≡ input8.txt 1 program PetikTest; 2 var 3 s: string; 4 begin 5 s := 'Baca buku ''KBBI'''; 6 s := ''; 7 s := ''; 8 writeln(s); 9 end.</pre>	<pre>1 PROGRAMSY 2 IDENT (PetikTest) 3 SEMICOLON 4 VARSY 5 IDENT (s) 6 COLON 7 IDENT (string) 8 SEMICOLON 9 BEGINSY 10 IDENT (s) 11 BECOMES 12 STRING ('Baca buku ''KBBI'') 13 SEMICOLON 14 IDENT (s) 15 BECOMES 16 CHARCON ('') 17 SEMICOLON 18 IDENT (s) 19 BECOMES 20 STRING ('') 21 SEMICOLON 22 IDENT (writeln) 23 LPARENT 24 IDENT (s) 25 RPARENT 26 SEMICOLON 27 ENDSY 28 PERIOD 29</pre>
Keterangan	
Pengujian ini dilakukan menggunakan program Arion yang memuat beberapa bentuk string dan karakter yang melibatkan tanda petik tunggal, yaitu string biasa, karakter petik tunggal, dan string kosong. Tujuan pengujian ini adalah untuk memastikan bahwa lexer mampu mengenali token string dan charcon dengan benar, termasuk pada kasus penggunaan escape tanda petik tunggal di dalam string maupun karakter.	

3.9. Pengujian Kesembilan

Input	Output
-------	--------

test > milestone1 > input > input9.txt

```
1  program FungsiTest;
2
3  const
4  MAX := 100;
5
6  type
7  Bilangan: integer;
8  DataAngka: record
9  nilai: integer;
10 end;
11
12 var
13 hasil: integer;
14
15 function tambah(a, b: integer): integer;
16 begin
17 tambah := a + b;
18 end;
19
20 procedure cetak(n: integer);
21 begin
22 writeln(n);
23 end;
24
25 begin
26 hasil := tambah(10, 20);
27 cetak(hasil);
28 end.
```

```
1  PROGRAMSY
2  IDENT (FungsiTest)
3  SEMICOLON
4  CONSTSY
5  IDENT (MAX)
6  BECOMES
7  INTCON (100)
8  SEMICOLON
9  TYPESY
10 IDENT (Bilangan)
11 COLON
12 IDENT (integer)
13 SEMICOLON
14 IDENT (DataAngka)
15 COLON
16 RECORDSY
17 IDENT (nilai)
18 COLON
19 IDENT (integer)
20 SEMICOLON
21 ENDSY
22 SEMICOLON
23 VARSY
24 IDENT (hasil)
25 COLON
26 IDENT (integer)
27 SEMICOLON
28 FUNCTIONSY
29 IDENT (tambah)
30 LPARENT
31 IDENT (a)
32 COMMA
33 IDENT (b)
34 COLON
35 IDENT (integer)
36 RPARENT
37 COLON
38 IDENT (integer)
39 SEMICOLON
40 BEGINSY
41 IDENT (tambah)
42 BECOMES
43 IDENT (a)
44 PLUS
45 IDENT (b)
46 SEMICOLON
47 ENDSY
48 SEMICOLON
49 PROCEDURESY
50 IDENT (cetak)
```

	<pre> 51 LPARENT 52 IDENT (n) 53 COLON 54 IDENT (integer) 55 RPARENT 56 SEMICOLON 57 BEGINSY 58 IDENT (writeln) 59 LPARENT 60 IDENT (n) 61 RPARENT 62 SEMICOLON 63 ENDSY 64 SEMICOLON 65 BEGINSY 66 IDENT (hasil) 67 BECOMES 68 IDENT (tambah) 69 LPARENT 70 INTCON (10) 71 COMMA 72 INTCON (20) 73 RPARENT 74 SEMICOLON 75 IDENT (cetak) 76 LPARENT 77 IDENT (hasil) 78 RPARENT 79 SEMICOLON 80 ENDSY 81 PERIOD </pre>
Keterangan	
Pengujian ini dilakukan menggunakan program Arion yang memuat deklarasi const, type, var, function, procedure, dan record. Tujuan pengujian ini adalah untuk memastikan bahwa lexer mampu mengenali keyword-keyword tersebut dengan benar, serta tetap dapat mengenali identifier, konstanta integer, operator assignment, operator penjumlahan, tanda kurung, koma, titik dua, titik koma, dan titik sesuai dengan token yang semestinya.	

3.10. Pengujian Kesepuluh

Input	Output
<pre>test > milestone1 > input > ≡ input10.txt 1 program ErrorTest; 2 var 3 s: string; 4 begin 5 s := 'teks belum ditutup; 6 writeln(s); 7 { komentar juga belum ditutup 8 end.</pre>	<pre>1 PROGRAMSY 2 IDENT (ErrorTest) 3 SEMICOLON 4 VARSY 5 IDENT (s) 6 COLON 7 IDENT (string) 8 SEMICOLON 9 BEGINSY 10 IDENT (s) 11 BECOMES 12</pre>
Keterangan Pengujian ini dilakukan menggunakan program Arion yang sengaja memuat kondisi tidak valid, yaitu string yang tidak ditutup dan komentar yang tidak ditutup. Tujuan pengujian ini adalah untuk memastikan bahwa lexer tidak hanya mampu mengenali token-token yang valid, tetapi juga dapat menangani kondisi kesalahan input dengan benar sesuai mekanisme error handling yang telah dirancang.	

3.11. Pengujian Kesebelas

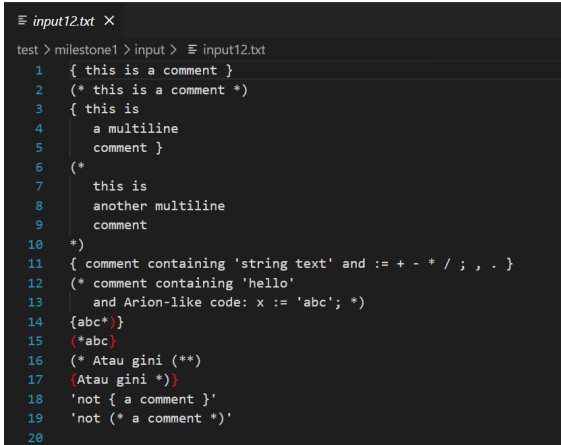
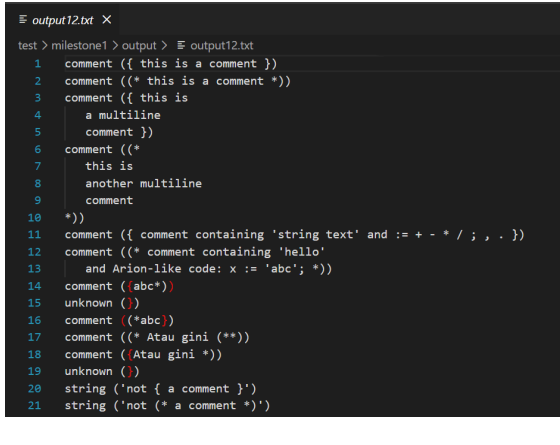
Input	Output
<pre>≡ input11.txt × test > milestone1 > input > ≡ input11.txt 1 program ErrorTest; 2 var 3 s: string; 4 begin 5 s := 'teks belum ditutup; 6 writeln(s); 7 { komentar juga belum ditutup 8 end.</pre>	Output/Error Lexical error terdeteksi. Program berhenti dengan exit code non-zero.
Keterangan Pengujian ini digunakan untuk menguji kemampuan lexer dalam mendeteksi lexical error pada literal string dan komentar yang tidak ditutup. Pada input ini terdapat string yang diawali dengan	

tanda petik tunggal, tetapi tidak memiliki pasangan penutup yang valid. Selain itu, terdapat komentar yang diawali dengan delimiter komentar, tetapi tidak ditutup hingga akhir file.

Kasus ini penting karena lexer tidak boleh tetap menghasilkan token normal ketika menemukan string atau komentar yang belum selesai. Apabila lexer tetap menerima input seperti ini, maka tahap parser dapat menerima token yang salah dan menghasilkan analisis sintaks yang tidak valid. Oleh karena itu, program diharapkan menghentikan eksekusi dan menampilkan pesan lexical error.

Hasil pengujian menunjukkan bahwa program berhasil mendeteksi input tidak valid tersebut. Program menghasilkan pesan error pada terminal dan mengembalikan exit code non-zero. Dengan demikian, error handling pada lexer sudah berjalan sesuai kebutuhan, terutama untuk kasus literal atau komentar yang tidak lengkap.

3.12. Pengujian Keduabelas

Input	Output
 <pre> test > milestone1 > input > E input12.txt 1 { this is a comment } 2 (* this is a comment *) 3 { this is 4 a multiline 5 comment } 6 (* 7 this is 8 another multiline 9 comment 10 *) 11 { comment containing 'string text' and := + - * / ; , . } 12 (* comment containing 'hello' 13 and Arion-like code: x := 'abc'; *) 14 {abc*} 15 (*abc) 16 (* Atau gini (** 17 Atau gini *) 18 'not { a comment }' 19 'not (* a comment *)' 20 </pre>	 <pre> test > milestone1 > output > E output12.txt 1 comment ({ this is a comment }) 2 comment ((* this is a comment *)) 3 comment ({ this is 4 a multiline 5 comment }) 6 comment ((* 7 this is 8 another multiline 9 comment 10 *)) 11 comment ({ comment containing 'string text' and := + - * / ; , . }) 12 comment ((* comment containing 'hello' 13 and Arion-like code: x := 'abc'; *)) 14 comment ({abc*}) 15 unknown () 16 comment ((*abc)) 17 comment ((* Atau gini (**)) 18 comment (Atau gini *)) 19 unknown () 20 string ('not { a comment }') 21 string ('not (* a comment *)') </pre>
Keterangan Pengujian ini bertujuan untuk memeriksa kemampuan lexer dalam mengenali komentar dan string yang mengandung simbol-simbol khusus. Input ini mencakup komentar satu baris, komentar multiline, komentar yang berisi simbol operator seperti :=, <, +, -, *, /, tanda baca, serta potongan teks yang menyerupai source code Arion. Selain komentar valid, pengujian ini juga memuat beberapa bentuk delimiter komentar yang tidak seimbang. Bagian tersebut digunakan untuk memastikan lexer dapat membedakan antara komentar yang benar-benar valid dan karakter penutup komentar yang muncul tanpa pasangan pembuka yang sesuai. Hasil output menunjukkan bahwa komentar valid berhasil dikenali sebagai token comment. Sementara itu, delimiter yang tidak sesuai menghasilkan token unknown dan diperlakukan sebagai lexical error oleh program utama. Pengujian ini membuktikan bahwa lexer mampu	

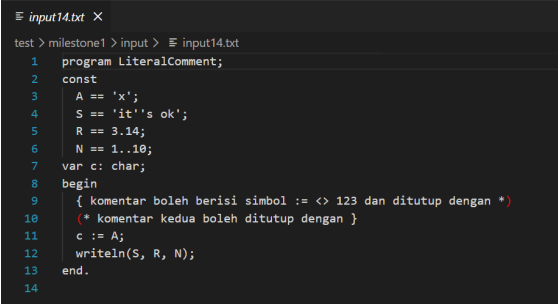
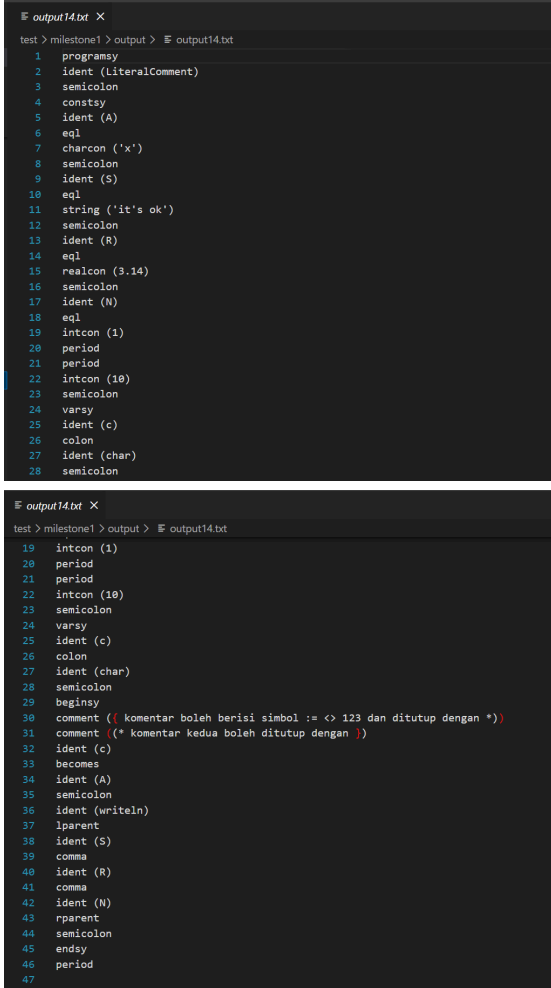
mengenali isi komentar tanpa memecahnya menjadi token-token lain, sekaligus tetap mendeteksi delimiter komentar yang tidak valid.

3.13. Pengujian Ketigabelas

Input	Output
<pre> ≡ input13.txt X test > milestone1 > input > ≡ input13.txt 1 PROGRAM KeywordBoundary; 2 VAR arrayzz, x2, x: INTEGER; 3 BEGIN 4 x := 5; 5 arrayzz := x DIV 2 MOD 1; 6 IF NOT (x >= 3) OR x <> 4 THEN 7 writeln('keyword boundary ok'); 8 END. 9 </pre>	<pre> ≡ output13.txt X test > milestone1 > output > ≡ output13.txt 1 programsy 2 ident (KeywordBoundary) 3 semicolon 4 varsy 5 ident (arrayzz) 6 comma 7 ident (x2) 8 comma 9 ident (x) 10 colon 11 ident (INTEGER) 12 semicolon 13 beginsy 14 ident (x) 15 becomes 16 intcon (5) 17 semicolon 18 ident (arrayzz) 19 becomes 20 ident (x) 21 idiv 22 intcon (2) 23 imod 24 intcon (1) 25 semicolon 26 ifsy 27 notsy 28 lparent 29 ident (x) 30 geq 31 intcon (3) 32 rparent 33 orsy 34 ident (x) 35 neq 36 intcon (4) 37 thensy 38 ident (writeln) 39 lparent 40 string ('keyword boundary ok') 41 rparent 42 semicolon 43 endsy 44 period 45 </pre>
Keterangan	
Pengujian ketigabelas digunakan untuk menguji batas antara keyword dan identifier. Input ini memuat keyword dengan variasi huruf besar seperti PROGRAM, VAR, dan BEGIN, serta identifier yang memiliki prefix menyerupai keyword, seperti arrayzz. Kasus ini penting karena lexer harus dapat membedakan keyword asli dari identifier yang hanya memiliki awalan mirip keyword. Sebagai contoh, kata array harus dikenali sebagai keyword arraysy apabila berdiri sendiri. Namun, kata arrayzz tidak boleh dikenali sebagai keyword karena merupakan identifier yang berbeda. Demikian juga identifier seperti x2 harus tetap dikenali sebagai ident, bukan dipecah menjadi ident dan intcon.	

Pengujian ini juga mencakup operator logika dan relasi seperti DIV, MOD, NOT, OR, >=, dan <. Hasil output menunjukkan bahwa lexer berhasil mengklasifikasikan keyword, identifier, operator, dan literal string dengan benar. File trace13.txt juga digunakan untuk menunjukkan proses transisi DFA selama lexer membaca karakter input satu per satu.

3.14. Pengujian Keempatbelas

Input	Output
 <pre> 1 program LiteralComment; 2 const 3 A == 'x'; 4 S == 'it's ok'; 5 R == 3.14; 6 N == 1..10; 7 var c: char; 8 begin 9 { komentar boleh berisi simbol := <> 123 dan ditutup dengan *} 10 (* komentar kedua boleh ditutup dengan *) 11 c := A; 12 writeln(S, R, N); 13 end. 14 </pre>	 <pre> 1 programsy 2 ident (LiteralComment) 3 semicolon 4 constsy 5 ident (A) 6 eql 7 charcon ('x') 8 semicolon 9 ident (S) 10 eql 11 string ('it's ok') 12 semicolon 13 ident (R) 14 eql 15 realcon (3.14) 16 semicolon 17 ident (N) 18 eql 19 intcon (1) 20 period 21 period 22 intcon (10) 23 semicolon 24 varsy 25 ident (c) 26 colon 27 ident (char) 28 semicolon 29 beginsy 30 comment ((komentar boleh berisi simbol := <> 123 dan ditutup dengan *)) 31 comment ((* komentar kedua boleh ditutup dengan *)) 32 ident (c) 33 becomes 34 ident (A) 35 semicolon 36 ident (writeln) 37 lparent 38 ident (S) 39 comma 40 ident (R) 41 comma 42 ident (N) 43 rparent 44 semicolon 45 endsy 46 period 47 </pre>
Keterangan Pengujian ini bertujuan untuk menguji pengenalan literal dan komentar pada lexer. Input ini mencakup character constant, string dengan tanda petik di dalamnya, real constant, range bilangan, identifier, dan komentar yang berisi simbol-simbol operator maupun tanda baca. Pada bagian literal, lexer diuji untuk mengenali token charcon, string, realcon, dan intcon secara tepat. String seperti 'it's ok' digunakan untuk memastikan lexer tetap dapat memproses isi string	

yang memiliki tanda petik sebagai bagian dari nilai string. Selain itu, ekspresi range seperti 1..10 digunakan untuk memastikan lexer tidak salah mengenali titik pada range sebagai bagian dari real number.

Pada bagian komentar, lexer diuji agar tidak memecah isi komentar menjadi token-token lain, meskipun di dalam komentar terdapat simbol seperti `:=`, `<>`, angka, dan keyword. Hasil pengujian menunjukkan bahwa lexer berhasil mengenali literal, komentar, operator, dan delimiter dengan benar. File `trace14.txt` memperlihatkan jejak transisi DFA ketika memproses literal dan komentar.

3.15. Pengujian Kelimabelas

Input	Output
<pre> ≡ input15.txt X test > milestone1 > input > ≡ input15.txt 1 program Bad; 2 var x: integer; 3 begin 4 x := 12abc; 5 y := 'unterminated 6 end. 7 </pre>	<pre> ≡ output15.txt X test > milestone1 > output > ≡ output15.txt 1 programsy 2 ident (Bad) 3 semicolon 4 varsy 5 ident (x) 6 colon 7 ident (integer) 8 semicolon 9 beginsy 10 ident (x) 11 becomes 12 unknown (12abc) 13 semicolon 14 ident (y) 15 becomes 16 unknown ('unterminated 17 end) 18 period 19 </pre>
Keterangan	
Pengujian kelimabelas digunakan untuk menguji deteksi lexical error pada gabungan angka dan huruf serta string yang tidak ditutup. Pada input ini terdapat lexeme 12abc, yaitu rangkaian karakter yang diawali angka tetapi dilanjutkan dengan huruf. Berdasarkan aturan token, bentuk tersebut tidak dapat diklasifikasikan sebagai integer constant maupun identifier yang valid. Selain itu, input juga mengandung string yang diawali tanda petik tunggal tetapi tidak memiliki penutup yang valid sebelum akhir input. Kasus ini digunakan untuk memastikan lexer mampu mendeteksi literal string yang tidak lengkap. Hasil pengujian menunjukkan bahwa lexer menghasilkan token unknown untuk lexeme yang tidak valid dan program utama memperlakukannya sebagai lexical error. Program kemudian berhenti dengan exit code non-zero. File <code>trace15.txt</code> digunakan untuk memperlihatkan proses transisi DFA sampai error ditemukan. Dengan demikian, pengujian ini membuktikan bahwa lexer tidak hanya mampu mengenali token valid, tetapi juga mampu menolak input yang melanggar aturan leksikal.	

3.16. Pengujian Otomatis

Selain pengujian manual melalui screenshot, implementasi terbaru juga menyediakan script regression test untuk Milestone 1. Script ini menjalankan seluruh testcase input1.txt sampai input15.txt, membandingkan output token aktual dengan expected output, serta memastikan testcase error menghasilkan exit code non-zero.

Perintah pengujian:

```
make all
./test/milestone1/run_milestone1_tests.sh
```

Hasil pengujian:

Milestone 1 lexer regression tests passed (12 valid + 3 invalid).

Pengujian otomatis ini memastikan bahwa perubahan pada lexer tidak merusak hasil tokenisasi yang sudah benar sebelumnya. Selain itu, testcase input13.txt, input14.txt, dan input15.txt juga menghasilkan DFA trace untuk membuktikan proses pembacaan karakter dan transisi state.

BAB IV

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa lexical analyzer untuk bahasa Arion telah berhasil dibuat sesuai tujuan Milestone 1. Program mampu membaca source code dalam bentuk file .txt, memproses input karakter demi karakter menggunakan DFA, dan menghasilkan daftar token dalam format .txt.

Lexer telah mampu mengenali token-token utama bahasa Arion, seperti keyword, identifier, integer constant, real constant, character constant, string, operator aritmatika, operator logika, operator relasi, assignment operator, comment, serta delimiter seperti tanda kurung, koma, titik koma, titik, dan titik dua. Program juga menerapkan prinsip longest match sehingga operator multi-karakter seperti `:=`, `<=`, `>=`, `<>`, dan `==` dapat dikenali sebagai satu token yang tepat.

Implementasi terbaru juga memperkuat pembuktian kesesuaian DFA melalui fitur DFA trace. Dengan fitur ini, proses pembacaan karakter, perpindahan state, dan pembentukan token dapat ditelusuri secara eksplisit. Selain itu, error handling telah diperbaiki sehingga input tidak valid seperti karakter tidak dikenal, string yang tidak ditutup, character literal yang tidak valid, dan komentar yang tidak ditutup akan menghasilkan pesan error informatif serta exit code non-zero.

Dengan demikian, implementasi Milestone 1 telah memenuhi kebutuhan utama lexical analysis, yaitu membaca input source code, mengklasifikasikan token dengan benar, menangani error leksikal, dan menyediakan output token yang dapat digunakan sebagai masukan untuk tahap syntax analysis pada Milestone 2.

4.2. Saran

Berdasarkan pengerjaan milestone 1 ini, terdapat beberapa saran untuk perbaikan dan pengembangan berikutnya di antaranya adalah :

- Perlunya memastikan kelengkapan dan kejelasan dalam pemetaan token pada *Deterministic Finite Automata* (DFA) di tahap spesifikasi. Dengan pemetaan yang komprehensif, seluruh jenis token dapat dikenali secara tepat sehingga dapat meminimalkan kebutuhan revisi terhadap aturan transisi antar state maupun diagram DFA yang telah dirancang.
- Selain itu, diperlukan peningkatan dalam manajemen waktu serta komunikasi selama proses pengembangan. Pengelolaan waktu yang lebih terencana dan komunikasi yang efektif antar anggota tim akan membantu
- Disarankan adanya proses *review* secara berkala terhadap desain DFA. Dengan melakukan validasi bersama, kesalahan dapat dideteksi lebih awal sebelum implementasi lebih lanjut dilakukan.

LAMPIRAN

Link GitHub: <https://github.com/alyanrrhma/MAR-Tubes-IF2224-2026>

Link Workspace Diagram: [Diagram DFA](#)

Tabel Pembagian Tugas:

No.	NIM - Nama	Tugas	Persentase
1.	13524017 - Aziza Dharma Putri	1. Membuat file token.cpp, token.hpp, Makefile 2. Membuat laporan 3. <i>Testing</i>	25%
2.	13524053 Ahmad Zaky Robbani	1. Membuat file config-lexer.txt dan main.cpp 2. Membuat laporan 3. Membuat diagram DFA versi revisi tambahan	25%
3.	13524063 - Marcel Luther Sitorus	1. Membuat dfa.cpp dan dfa.hpp 2. Membuat laporan 3. Debugging	25%
4.	13524081 - Alya Nur Rahmah	1. Membuat lexer.cpp dan lexer.hpp 2. Membuat laporan 3. <i>Testing</i>	25%

DAFTAR PUSTAKA

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson Education.

TutorialsPoint. (n.d.). *Compiler Design - Lexical Analysis*. Accessed online from https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Pearson Education. Accessed online from [https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20(2006).pdf)

Free Software Foundation. (n.d.). *An Introduction to Makefiles*. Accessed online from https://www.gnu.org/software/make/manual/html_node/Introduction.html